

Edge cases (a task that needs no tool at all) — Hints

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 2 depth levels: **Basic** (the plain idea) and **Intermediate** (a real check, not eyeballing it). Read Basic first even if this feels obvious — it's the fastest way to spot exactly what Intermediate adds.

- [Hint 1 — The idea, and what to check](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — The idea, and what to check

Basic Version

Ask your agent something it can answer from general knowledge alone — something none of your registered tools are for. Something like "What's the capital of France?" when your only tools are `get_weather` and `celsius_to_fahrenheit`. Run it through your `run_agent()` from the previous exercise, and check: did it call a tool at all, or did it just answer?

Intermediate Version

"Check" here means a real assertion, not reading the printed output and deciding it looks fine. Your `run_agent()` from `build_react_loop` returns an `AgentResult` with a `steps` list — for a task that needs no tool, that list should come back **empty**, because the very first model response should already be a final answer with no `tool_calls`. Write this as a real check: `assert len(result.steps) == 0`, not something you eyeball in a terminal and move past.

Worth knowing as you write this: "the model didn't need a tool" is a decision the model makes on its own, based on the tool *descriptions* you wrote in Doc06, and it isn't 100% guaranteed to go the way you expect. A vaguely-worded tool description (e.g., `get_weather`'s docstring just says "get information about a city" instead of "get the current weather for a city") can make the model reach for it even for "what's the capital of France?" — that's a real bug this exercise is partly designed to catch, by choosing your test prompts to be unambiguously answerable without any tool.

Difference between Basic and Intermediate: Basic picks an obviously-safe prompt and eyeballs the result. Intermediate turns that into a real, automatable assertion on the `steps` list — this is the depth the exercise's Solution is written at.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
pick a prompt with no matching tool, e.g. "What's the capital of France?"

result = run_agent(prompt)

print result.final_answer
print result.steps          -- should be empty
```

Here's almost the whole thing — fill in your own prompt:

```
# no_tool_needed_practice.py
result = run_agent("...", max_iterations=5)
print(result.final_answer)
print(result.steps)
```

Expected output if the loop is working correctly: the answer, and an empty list — `[]` — for `result.steps`, since no tool was ever called.

Intermediate Version

```
function test_no_tool_call_for_general_knowledge():
    result = run_agent("What's the capital of France?")
    assert result.steps is empty, "expected zero tool calls"
    assert result.final_answer mentions "Paris"
```

```
# no_tool_needed_practice.py
def test_no_tool_call_for_general_knowledge():
    result = run_agent("What's the capital of France?", max_iterations=5)
    assert len(result.steps) == 0
    assert "Paris" in result.final_answer
```

Run this a few times in a row — LLM output isn't perfectly deterministic, so it's worth seeing it pass more than once before trusting it. Write a second test for a second no-tool prompt of your own before checking the Solution.

Difference between Basic and Intermediate: Basic prints the result and reads it by eye. Intermediate turns it into a real, repeatable `assert`-based test — this is the depth the exercise's Solution is written at.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)